

Design Doc: Async Export Pipeline

Background

The reporting service currently generates CSV exports synchronously inside the request handler, which means a large export can hold an HTTP connection open for over a minute and regularly trips the load balancer's 30-second timeout. Support has logged this as the single most common cause of export-related tickets over the last two quarters, and it gets worse as customer data volume grows, not better.

This document proposes moving export generation to an asynchronous job queue, with the client polling a status endpoint until the export is ready to download.

Requirements

REQ-101: An export request must return immediately with a job identifier, not block until the export file is fully generated.

REQ-102: A client must be able to poll job status and retrieve a download URL once the job completes, without any change to authentication already in place.

REQ-103: A failed export job must surface a clear, actionable error to the client, not a silent timeout indistinguishable from still-processing.

Job Queue Design

Decision: jobs are queued in the existing background-worker infrastructure already used for scheduled reports, rather than introducing a new queue technology. Reusing existing infrastructure was chosen over a dedicated export queue because the operational cost of running and monitoring a second queue system was judged not to be justified by export jobs alone.

Supported job states:

- queued
- running
- complete
- failed

Rollout Steps

1. Ship the async endpoint behind a feature flag, default off
2. Enable for internal accounts first and monitor job latency
3. Roll out to all accounts once queue depth stays stable for a week

Job State Comparison

State	Client action	Terminal
queued	keep polling	No
running	keep polling	No
complete	download via URL	Yes
failed	show error, allow retry	Yes

Reference Implementation Sketch

```
def enqueue_export(account_id, filters):
    job_id = queue.push(ExportJob(account_id, filters))
    return job_id
```

Failure Modes

WARNING: a client that never polls for status will never learn its export failed. The design must still email a failure notification for jobs older than an hour with no successful poll, so a failure is never silently lost.

NOTE: job records are retained for 24 hours after completion, then purged — this is a deliberate storage-cost trade-off, not an oversight; a client that waits longer than that to download must re-request the export.

TODO: determine the exact retry policy for a transiently-failed job once the async path is live in production — the current single-attempt behavior is a starting point, not a measured decision.

Open Questions

Should very small exports (under some row-count threshold) still be generated synchronously, to avoid the round-trip latency of polling for something that would have completed in under a second anyway? Current lean is no, for consistency of client implementation, but this needs data from the initial internal rollout before being decided for good.

Related Work and Prior Art

Several other teams at the company have faced a version of this same long-request problem before, and it is worth summarizing what they did and why their solutions do not directly transfer here, so reviewers of this document do not spend time re-suggesting approaches that were already considered and set aside for reasons specific to this system.

The billing team's statement-generation pipeline faced a superficially similar long-request problem two years ago, where generating a customer's annual statement could take well over a minute for accounts with a large transaction history. Their solution was to precompute statements on a nightly schedule rather than on demand, trading immediacy for a bounded, predictable request latency. That approach was considered here and rejected, because export requests in this system are triggered on demand with arbitrary, user-chosen filters, so there is no fixed nightly job that could precompute every possible export a client might eventually ask for.

The analytics team's dashboard-export feature took a different approach that is closer in spirit to what this document proposes: they built a polling endpoint backed by an in-memory job table, without any durable queue behind it at all. This is a strictly narrower solution than what is proposed here, because an in-memory job table does not survive a process restart, and their product surface tolerates an occasional lost job in a way that a paying customer's export request should not. Their approach was considered as a smaller, lower-risk first step, but ultimately rejected as the primary design here because durability across a deploy or restart is not negotiable for this system in the way it was for theirs.

It is also worth noting that several commercial export-as-a-service products this company evaluated in the past offer asynchronous export generation as a built-in feature, and part of the motivation for building this internally rather than adopting one of those products wholesale is that the existing reporting service is deeply integrated with several other internal systems in ways that would make a wholesale platform migration a much larger project than the narrower async-export problem this document is trying to solve. This document does not attempt to relitigate that broader platform decision, which was made independently and is out of scope here; it simply takes the existing reporting service architecture as a given constraint and designs within it.

Long-Term Considerations

As export volume continues to grow, it is worth thinking ahead about what happens if the background-worker infrastructure's own capacity becomes a constraint in its own right, independent of the request-latency question this document has focused on so far. Today's estimated export volume comfortably fits within the worker infrastructure's existing capacity, but there is no guarantee that remains true as the customer base grows over the next several quarters, and this document does not attempt to model that growth curve precisely.

One option, if worker capacity does eventually become a real constraint, is to introduce a dedicated worker pool for export jobs specifically, separate from the pool scheduled reports already share, so a burst of export activity cannot delay scheduled reports and vice versa. This was considered and deliberately deferred rather than built upfront, since introducing a second worker pool before there is real evidence of contention would add operational complexity without a demonstrated need for it. Another consideration for the long term is whether the job-state model described in the comparison table above will need to grow beyond its current four states if a future requirement introduces something like a partially complete export, where some requested data is ready and some is not. This document does not attempt to anticipate that case, but the job-state model was chosen specifically as an easily-extensible enum rather than a boolean flag, precisely so a future state could be added without requiring a full redesign of the polling contract.

Finally, this document has focused entirely on the client-facing polling contract and has said comparatively little about the worker side's own failure modes beyond the high-level retry policy already flagged as an open question above. A more detailed treatment of worker-side failure modes, including what happens if a worker crashes mid-export after partially writing output, is deferred to a follow-up document once the basic polling contract has been validated in production against real traffic, rather than attempting to anticipate every worker-side edge case speculatively before any real operational experience exists.

A related, still-open question is how export jobs should interact with the reporting service's existing rate limiting, which today is applied only to synchronous request paths and has never needed to account for a background job that can legitimately run for several minutes. Naively applying the same per-minute request-count limit to job submissions would likely be too permissive, since a client could submit far more expensive work per request than the synchronous limit was ever calibrated to bound, while applying no limit at all would remove a safeguard the platform team has relied on for years to protect shared infrastructure from a single misbehaving client. Resolving this properly will likely require a job-cost estimate that factors in the size of the filtered dataset, not just a flat per-request count, which is a meaningfully larger piece of work than anything else described in this document and is deliberately not attempted here.

A second, smaller open question is whether the polling endpoint should support a long-polling mode, where the server holds the connection open for a bounded window and responds immediately if the job completes within it, rather than requiring the client to issue a fresh request on a fixed interval regardless of how close the job is to finishing. Long polling would reduce the total number of requests a client issues for a typical export, at the cost of holding more connections open on the reporting service for longer than a simple fixed-interval poll would, which is exactly the kind of connection-longevity tradeoff this whole design was originally meant to move away from for the synchronous export path. Whether that tradeoff is still worth making for the polling endpoint specifically, where the held-open window would be much shorter and bounded rather than open-ended, is left for a follow-up decision once real polling-frequency data exists from the initial rollout, and is called out here mainly so it is not forgotten entirely once the simpler fixed-interval version ships and appears to work well enough that nobody feels urgency to revisit it.